Ordinary for This Moment? Generator-Conditional Null Models for Limit Order Book Surveillance

Arshad Khatib

***Abstract*—Surveillance systems flag order flow that is unusual, but unusual relative to a pooled history confuses busy moments with manipulation. We test whether a window of level-1 order book events is ordinary for its own moment. A fitted conditional generator simulates continuations of the window from its real context, Monte Carlo p-values of seven statistics are combined, and split-conformal calibration on clean windows turns the score into a p-value that is valid under exchangeability. On validation days of six Binance perpetual futures, with spoofing, quote stuffing and momentum ignition injected at 13 strengths, we compare this detector with one that differs only in using an unconditional null. The pre-registered comparison returned NO-GO: the conditional detector had more power in all three families, but its false-alarm rate at $\alpha = 0.05$ reached 0.110 on one symbol, above the 0.10 limit. A follow-up, also fixed in advance, traced this to adaptive Monte Carlo refinement triggered by injected copies and, mostly, to drift from the calibration day. With refinement matched and each day calibrated on the day before, the largest false-alarm rate fell to 0.075 and power remained higher in all three families. A confirmatory run of this design on six unread test days, with its rule committed beforehand, returned NOT CONFIRMED (validity): the unconditional detector exceeded the false-alarm limit on two symbols, while the conditional detector stayed between 0.039 and 0.052 on all six. At matched false-alarm rates its power was higher for spoofing and quote stuffing on every symbol, but not for momentum ignition. In an exploratory analysis, the unconditional false alarms came on days when the market became much busier than the calibration day.**

***Index Terms*—Market surveillance, limit order book, conformal p-values, spoofing, quote stuffing, generative models.**

I. Introduction

A surveillance system flags order flow that looks unusual, and the hard question is: unusual compared with what? A burst of quote updates is routine at a busy moment and suspicious at a quiet one. A detector that compares every window with one pooled distribution of "normal" windows treats the two cases alike. It raises alarms whenever the market is busy, and it misses manipulation that is large for a quiet moment but small for the day.

We ask instead whether a window is ordinary *for this moment*. A fitted level-1 order book generator simulates how the next 100 events could have unfolded, starting from the 100 real events that preceded the window and from the market's slow activity state. Each statistic of the observed window gets a Monte Carlo p-value against those simulations, and the p-values are combined into one score. Split-conformal calibration on real clean windows [1], [2] then turns the score into a p-value that is valid in finite samples if calibration and evaluation windows are exchangeable, whatever the generator gets wrong. The generator decides power; the calibration decides validity.

The generator comes from our companion study [3], which found that, in level-1 order flow, the realism of synthetic data depends on the noise model far more than on the model of the conditional mean. Here the generator is frozen and used for a different purpose: as the null model of a test.

We evaluate the idea on 6 Binance perpetual futures contracts, on validation days and then once on held-out test days, with labelled injections of three manipulation footprints that are visible at level 1: touch spoofing, quote stuffing and momentum ignition, each at several strengths. The comparison that matters is with an unconditional detector that uses the same statistics, the same combination and the same conformal calibration, and differs only in its null. Every decision rule was written down and committed before the analysis it governs.

The pre-registered comparison came out negative, and the reason is instructive. The conditional detector detected more injections in all three families, but its false-alarm rate on clean windows exceeded the limit we had set on one symbol, so the rule returned NO-GO. A follow-up, also fixed in advance, separates two explanations. With thresholds matched to equal false-alarm rates, the conditional detector still had higher power in all three families, so its advantage is not an artefact of a looser threshold. A rerun with the raw p-values saved showed that the excess false alarms came partly from an asymmetry in how the two window sets were scored and mostly from drift between the calibration day and the evaluation days. With both repaired, every symbol stayed within the limit and the conditional detector kept higher power in all three families. The repair was chosen on the same validation days, so we ran it once on the six unread test days under a rule committed beforehand. That run is not a confirmation. The conditional detector stayed within the false-alarm limit on every symbol and every day, but the unconditional detector did not, and the rule required both; the power advantage held for two families and reversed for the third.

Our contributions are:

- a surveillance test that uses a fitted conditional order book generator as its null and conformal calibration for validity;
- a paired evaluation in which the two detectors share statistics, calibration windows and injected windows, so that their difference isolates the null;
- two documented failure modes of conformal validity in this setting, adaptive Monte Carlo refinement triggered by a window's injected copies and calibration-day drift, with a repair that keeps the power advantage on validation days;

A. Khatib is with the Department of Artificial Intelligence, Sardar Vallabhbhai National Institute of Technology (SVNIT), Surat 395007, Gujarat, India (e-mail: u24ai112@aid.svnit.ac.in).

- a confirmatory test-day run of the repaired design, reported in full although it did not confirm;
- code, result files and every decision rule, public at https://github.com/Arshad-13/lob-surveillance.

II. Related Work

Manipulation and its detection. Putniņš [4] surveys manipulation schemes and the evidence on them. Cartea, Jaimungal and Wang [5] model spoofing as a strategy that shifts the displayed queue imbalance to move other traders' beliefs, and Lee, Eom and Park [6] study the strategic behaviour and performance of spoofing traders. In the agent-based market of Wang and Wellman [7], traders who learn from the order book can be spoofed, and spoofing lowers total surplus. Egginton, Van Ness and Van Ness [8] find quote stuffing episodes in over 74% of US exchange-listed securities in 2010, with lower liquidity, higher trading costs and higher short-term volatility during the episodes.

Most detectors learn from labelled or weakly labelled manipulation, or model the manipulation itself. Cao et al. [9] fit a hidden Markov model with anomaly states to wavelet and gradient features of tick prices. Tao et al. [10] derive the optimal strategy of a potential spoofer and apply it to Level-2 data from the TMX. Kularatnam and Stathaki [11] label order book sequences algorithmically, train a temporal convolutional network on the labels and rank new cases by similarity to expert-reviewed ones; Lin and Yang [12] learn contrastive representations for multilevel manipulation; and Fabre and Challet [13] learn how "spoofable" a book state is with interpretable probabilistic networks. Tiwari, Ramampiaro and Langseth [14] survey machine learning for market surveillance more broadly. Learned detectors need labelled manipulation, which is scarce and specific to the schemes it contains; our test needs only clean data.

Order book generators as simulators. Coletta et al. use conditional generative adversarial networks as reactive market simulators [15] and study their robustness [16]. We use a generator as a statistical null rather than as an environment, which shifts the requirement from "realistic on average" to "calibrated in the tail for the given context".

Conformal outlier detection. Split-conformal p-values [1] are valid in finite samples under exchangeability, and Bates et al. [2] show that they are positively dependent in the way that Benjamini–Hochberg [17] needs, so false-discovery control carries over. When calibration and test data are not exchangeable, weighting [18], [19] or online adjustment of the level [20] restores approximate coverage. We combine dependent per-statistic p-values with the Cauchy combination [21], and compute Monte Carlo p-values with the $(1+\#)/(M+1)$ form, which is never zero [22].

In searches of OpenAlex on 21 and 24 October 2026 we found no work that uses a fitted conditional order book generator as the null distribution of a surveillance test, calibrates it conformally and evaluates it against labelled injections. We claim that combination and its evaluation, not any single ingredient.

TABLE I
The statistic panel. Same-price events leave both best prices unchanged.

Statistic	Running score
`range`	range of the cumulative mid-price change
`n_moves`	number of mid-price moves
`max_spread`	spread
`queue_swing`	absolute change of the log best-queue imbalance since the end of the context
`queue_jump`	absolute one-event change of a log best queue, at same-price events
`queue_roundtrip`	a same-price jump up in a best queue followed within 50 events by a jump down; the smaller of the two
`n_fast`	number of events in the same millisecond as the previous one

III. Method

A. Events and windows

An event is the level-1 book after one update, described by five features: the mid-price change in ticks, the spread in ticks, the log best-bid and best-ask sizes, and the log time since the previous event. A window is a stretch of 100 consecutive events, and its context is the 100 events before it. Windows do not overlap.

B. The conditional null

The generator [3] predicts the next event's mean with a ridge-fitted vector autoregression and the log size of its deviation from the mean with a second ridge readout. The second readout also sees three exponentially weighted averages of the activity (whether the mid moved, the spread, the log inter-event time) with half-lives of 10, 100 and 1,000 events. To generate an event, it finds the 64 moments in a pool of real moments whose predicted mean and scale are nearest to the current ones, picks one at random, and adds that moment's residual to the current mean. Both readouts are fitted on training days only. For a window starting at stream position s, the pool holds the 30,000 real moments immediately before s, so no simulation sees the window or anything after it. The configuration (variant `slow-s`, pool $W = 30{,}000$) was fixed in the companion study before its test run; we do not tune it here. Every result file stores a hash of the generator code that produced it.

For each window the generator draws M continuations of 100 events from the real context and the real activity averages at $s-1$. Injections touch only the observed window, never the stream the generator reads, so one set of continuations serves every injected copy of a window.

C. Statistics

Each statistic is the maximum over the window of a running score (Table I). The panel targets the three footprints and also responds to ordinary volatility, which is what the null must explain away.

D. Scores and p-values

For window w and statistic T_j, let $N_>$ and $N_=$ count the continuations $Y_1, \ldots, Y_M$ with $T_j(Y_m) > T_j(w)$ and $T_j(Y_m) = T_j(w)$. The conditional detector's p-value is the Monte Carlo tail probability

$$p_j(w) = \frac{N_> + U\,(1 + N_=)}{M + 1}, \tag{1}$$

with $U \sim \mathrm{U}(0, 1)$ breaking ties, which matters because several statistics are counts. The unconditional detector's $p_j(w)$ is the same expression with the continuations Y_m replaced by a fixed reference set of real clean windows. Both detectors combine the p_j by the Cauchy combination [21], with each p_j capped at 0.99 so that a statistic with no evidence (for example a count of zero) cannot swamp the others, and score the window by $g(w) = -\log p_{\mathrm{comb}}(w)$.

Conformal calibration. Given the scores $g_1, \ldots, g_n$ of n clean calibration windows, a window's conformal p-value is

$$\hat{p}(w) = \frac{1 + \#\{i : g_i \geq g(w)\}}{n + 1}. \tag{2}$$

If w is clean and exchangeable with the calibration windows, $\Pr(\hat{p}(w) \leq \alpha) \leq \alpha$, whatever model produced the scores [1], [2]. A bad generator therefore costs power, not validity, as long as the calibration windows are treated exactly like the windows being tested.

Refinement. The conditional p-value cannot go below $1/(M + 1)$, while the unconditional reference set has about 1500 windows. To give the two detectors comparable resolution in the tail, the Stage-0 design drew $M = 200$ continuations and, for any window in which some statistic reached $p_j \leq 2/(M+1)$, 1000 in all. Section V shows that the way this rule was applied broke exchangeability; the repaired design applies it to each copy of a window separately.

E. Injected footprints

Each family is injected into a real window starting 10 to 60 events in, with sizes in the symbol's own units from its training days. Inserted events take the training 1% quantile of the inter-event time and keep the quotes on the price grid.

- **Spoofing:** an order of k times the median best-queue size joins the bid or the ask at a same-price event, stays while that best price is unchanged and is cancelled after $\tau = 20$ events. If the price moves first, the order leaves the level-1 view without a cancel. $k \in \{5, 10, 20, 50, 100\}$.
- **Quote stuffing:** n fast events, each a one-lot flicker of a best queue with prices unchanged. $n \in \{10, 25, 50, 100\}$.
- **Momentum ignition:** m fast one-tick mid moves in one direction, and m moves back 10 events later. $m \in \{1, 2, 3, 5\}$.

Only the manipulator's events are added; the rest of the window is the real one. We do not model the market's reaction to the manipulation.

IV. Experimental Design

A. Data

We use Binance USD-M perpetual futures best-bid/ask updates (`bookTicker`, https://data.binance.vision) for BTC, ETH, SOL, XRP, DOGE and LTC against USDT, from 1 February to 1 March 2024, keeping the first 100,000 events after 14:00 UTC of each day. Days are split 18/6/6 into training, validation and test, as in the companion study. Stage 0 and the follow-up use training and validation days only. The test days were read once, by the confirmation run of Section IV-E.

B. Window sets

Per symbol, the six validation days give three disjoint sets:

- **A**, reference: 1500 windows from days 1–2, the unconditional detector's reference set;
- **B**, calibration: all 998 windows of day 3, clean, shared by both detectors;
- **E**, evaluation: 1500 windows sampled from days 4–6.

Each E window is scored clean and in 13 injected copies (5 spoofing, 4 stuffing and 4 ignition strengths). Within a family, all strengths share one position, side and direction, so the strengths are paired.

C. Measures

The *power* of a detector at level α is the share of injected copies with $\hat{p} \leq \alpha$; its *realised false-alarm rate* is the share of clean E windows with $\hat{p} \leq \alpha$. We use $\alpha = 0.05$ (primary) and 0.01, and average power over a family's strength grid and over the six symbols. With 1500 clean windows, the standard error of a false-alarm rate of 0.05 is 0.006.

D. Decision rules

The Stage-0 rule, written before the run, was: GO if the conditional detector has higher power at $\alpha = 0.05$ on at least two of the three families, and both detectors' realised false-alarm rate is at most $2\alpha = 0.10$ on every symbol. An earlier pilot had used Benjamini–Hochberg power at 5% prevalence as its primary measure; at a calibration size of 600 windows neither detector could make a single discovery, so we replaced the measure before this run.

After the NO-GO, we wrote a follow-up rule, again before any new analysis (Section V-B). Both rules, with their commit dates, are in the repository.

E. Test-day confirmation

The follow-up chose its repaired design on the same validation days on which it judged it. We therefore fixed one run of that design on the six test days, with its decision rule, and committed both before any test window was read. Per symbol, the reference set A holds up to 1,500 windows from validation days 4 and 5, the first calibration set is every clean window of validation day 6, and every window of test days T1–T6 is evaluated, clean and in the same 13 injected copies. Test day T1 is calibrated on validation day 6, and each later day on the clean windows of the day before. Resolution, statistics, combination and conformal p-values are those of the repaired design. The run is void unless the generator's files are byte-identical to those of Stage 0.

The rule has two parts. *Validity*: both detectors' realised false-alarm rate at $\alpha = 0.05$ is at most 0.10 on every symbol.

TABLE II
STAGE 0: POWER AT $\alpha = 0.05$, AVERAGED OVER EACH FAMILY'S STRENGTH GRID, AND THE REALISED FALSE-ALARM RATE ON CLEAN EVALUATION WINDOWS. COND.: GENERATOR-CONDITIONAL NULL; UNCOND.: UNCONDITIONAL NULL.

	BTC	ETH	SOL	XRP	DOGE	LTC	Mean
Spoofing, cond.	0.70	0.54	0.60	0.88	0.66	0.76	0.691
uncond.	0.33	0.21	0.51	0.70	0.30	0.44	0.414
Quote stuffing, cond.	0.19	0.21	0.29	0.24	0.31	0.40	0.275
uncond.	0.10	0.12	0.26	0.14	0.12	0.24	0.163
Momentum ignition, cond.	0.24	0.19	0.08	0.74	0.58	0.68	0.419
uncond.	0.05	0.04	0.10	0.57	0.11	0.45	0.219
False alarm, cond.	0.051	0.047	0.059	0.110	0.072	0.082	0.070
uncond.	0.027	0.025	0.079	0.035	0.026	0.041	0.039

Power: a family is confirmed if the pooled difference in power at $\alpha = 0.05$, conditional minus unconditional, has a 95% bootstrap interval above zero; the interval resamples test windows within each symbol, a window together with its injected copies, 2,000 times, and holds the calibration sets fixed. The outcome is CONFIRMED if validity holds and at least two families are confirmed, and NOT CONFIRMED, for validity or for power, otherwise.

F. Compute

The generator runs on CPU. Stage 0 ran the six symbols in parallel on a 16-core cluster node, 1.6–2.3 hours per symbol; the test-day run took 6.2–7.8 hours per symbol on the same kind of node.

V. RESULTS

A. Stage 0: more power, too many false alarms

Table II gives the pre-registered comparison. Averaged over strengths and symbols, the conditional detector's power at $\alpha = 0.05$ was 0.691 against 0.414 for spoofing, 0.275 against 0.163 for quote stuffing and 0.419 against 0.219 for momentum ignition. Its realised false-alarm rate, however, was 0.110 on XRP, above the 0.10 limit, and 0.072 and 0.082 on DOGE and LTC. The rule therefore returned NO-GO. The unconditional detector stayed at or below 0.05 on 5 of 6 symbols and reached 0.079 on SOL. Against the sampling tolerance of a rate of 0.05 measured on 1500 windows (0.05 plus two standard errors, 0.061), the conditional detector exceeded it on 3 symbols and the unconditional one on 1.

A false-alarm rate above α means that clean evaluation windows scored higher, relative to the calibration windows, than exchangeability allows. Part of the power advantage could then come from a looser effective threshold rather than from better ranking.

B. Follow-up

While writing the follow-up rule we noticed, in the run's metadata, that the refinement had not been applied alike to the two sets. It is decided per window, and in E a window counts as reaching the floor if *any* of its copies, clean or injected, does. Strong injections almost always reach it, so 96–100% of clean E windows were scored with 1000 continuations, against 7–13% of the calibration windows. The Monte Carlo p-value is more conservative at $M = 200$, so calibration scores sit lower than the scores of clean evaluation windows. Calendar drift between day 3 and days 4–6 is a second candidate. The follow-up asks two questions.

TABLE III
FOLLOW-UP Q1: POWER WITH EACH DETECTOR'S THRESHOLD SET SO THAT ITS FALSE-ALARM RATE ON CLEAN EVALUATION WINDOWS IS AT MOST α, AND THE AUC BETWEEN CLEAN AND INJECTED COPIES; MEANS OVER STRENGTHS AND SYMBOLS. BOLD: THE HIGHER OF THE PAIR.

	Matched, $\alpha = 0.05$		Matched, $\alpha = 0.01$		AUC	
Family	cond.	uncond.	cond.	uncond.	cond.	uncond.
Spoofing	**0.629**	0.498	0.140	**0.163**	**0.917**	0.855
Quote stuffing	**0.239**	0.191	0.042	**0.091**	**0.622**	0.567
Momentum ignition	**0.376**	0.259	**0.137**	0.101	**0.780**	0.714

Q1: does the advantage survive at matched size? For each detector and symbol we lower the threshold until its false-alarm rate on the clean E windows is at most α, and recompute power. This uses the saved p-values and needs no new run. Q1 holds if the conditional detector has higher size-matched power at $\alpha = 0.05$ on at least two families, with power at least as high on at least four of six symbols in each.

Q1 holds (Table III). The size-matched power is 0.629 against 0.498 for spoofing (conditional detector at least as high on 5 of 6 symbols), 0.239 against 0.191 for stuffing (6/6) and 0.376 against 0.259 for ignition (6/6). Matching the size shrinks the spoofing gap from 0.28 to 0.13 but does not close it, and the threshold-free AUC favours the conditional detector in all three families. At the stricter level $\alpha = 0.01$, which the rule treats as secondary, the picture splits: the conditional detector is ahead for ignition (0.137 against 0.101) but behind for spoofing (0.140 against 0.163) and stuffing (0.042 against 0.091).

Q2: can validity be repaired, and which cause was it? We reran Stage 0 with the same seed and settings, this time saving each window's per-statistic p-values from the first 200 continuations and from all of them, with the window's day. The rerun reproduced the saved conformal p-values exactly (maximum absolute difference 0 on every symbol), so the arms below rescore the Stage-0 windows rather than new ones. Three arms change the design. V1 scores a copy of a window with 1000 continuations only if that copy itself reaches the floor, which is the rule the calibration set already followed. V2 calibrates each evaluation day on the clean windows of the day before: day 4 on B, days 5 and 6 on the clean E windows of days 4 and 5 (494–531 windows). V3 does both. The unconditional detector is recalibrated on the same sets. The rule takes the first of V1 and V3 under which both detectors stay at or below 0.10 on every symbol.

Under V1 the share of clean E windows scored with 1000 continuations fell to 8–13%, in line with the calibration set, and the conditional false-alarm rate fell on XRP, DOGE and LTC (to 0.102, 0.059 and 0.072). XRP stayed above 0.10, so V1 does not qualify. V3 does: its largest false-alarm rate is 0.075 for the conditional detector and 0.062 for the unconditional one (Table IV). Under V3 the conditional detector's power at $\alpha = 0.05$ is 0.655 against 0.444 for

TABLE IV

FOLLOW-UP Q2: THE LARGEST REALISED FALSE-ALARM RATE OVER THE SIX SYMBOLS AND POWER AT $\alpha = 0.05$ (MEANS OVER STRENGTHS AND SYMBOLS) UNDER EACH ARM. UNDERLINED: ABOVE THE 0.10 LIMIT. BOLD: THE HIGHER POWER OF THE PAIR.

	Max. false alarm		Spoofing		Stuffing		Ignition	
Arm	cond.	uncond.	cond.	uncond.	cond.	uncond.	cond.	uncond.
Original	0.110	0.079	**0.691**	0.414	**0.275**	0.163	**0.419**	0.219
V1 matched resolution	0.102	0.079	**0.682**	0.414	**0.269**	0.163	**0.409**	0.219
V2 rolling calibration	0.077	0.062	**0.657**	0.444	**0.249**	0.173	**0.394**	0.235
V3 both	0.075	0.062	**0.655**	0.444	**0.248**	0.173	**0.390**	0.235

TABLE V

THE CONDITIONAL DETECTOR'S REALISED FALSE-ALARM RATE AT $\alpha = 0.05$ BY VALIDATION DAY, UNDER THE STAGE-0 DESIGN AND UNDER V3. DAY 4 IS CALIBRATED ON DAY 3 IN BOTH.

	Original			V3		
Day	4	5	6	4	5	6
BTC	0.049	0.059	0.046	0.049	0.059	0.040
ETH	0.040	0.049	0.053	0.040	0.051	0.044
SOL	0.064	0.047	0.067	0.064	0.026	0.067
XRP	0.104	0.095	0.133	0.096	0.065	0.063
DOGE	0.058	0.069	0.091	0.056	0.051	0.053
LTC	0.073	0.083	0.091	0.058	0.065	0.046

spoofing, 0.248 against 0.173 for stuffing and 0.390 against 0.235 for ignition. Since Q1 holds, the follow-up returns GO, and V3 replaces the Stage-0 design.

The GO comes with two qualifications. V3 meets the 0.10 tolerance, not nominal size: XRP's 0.075 for the conditional detector and SOL's 0.062 for the unconditional one exceed 0.061. And the repair narrows the spoofing gap from 0.28 to 0.21, because the conditional detector loses part of its looser threshold and the unconditional detector gains a little from the closer calibration day.

Attribution. Averaged over symbols, the conditional false-alarm rate was 0.070 under the original design, 0.065 under V1 and 0.055 under V3. Of the excess over 0.05, matched resolution removed about a quarter and day-before calibration about half. V2 alone also meets the tolerance (at most 0.077), so drift was the larger cause. Table V breaks the conditional rate down by day. Drift from the calibration day predicts a rise with distance from day 3. DOGE and LTC rise steadily and XRP reaches 0.133 on day 6, while BTC, ETH and SOL show no trend. Under V3 the later days come into line, but XRP's day 4, which both designs calibrate on the adjacent day 3, stays at 0.096. Distance in time does not explain everything; on XRP even consecutive days differ.

C. Test days

The confirmation run passed both integrity checks: the generator files were byte-identical to those of Stage 0, and the code files to those of the commit that fixed the rule. Each symbol contributed 5988 test windows (6.2–7.8 hours per symbol). Matched resolution behaved as designed: 9–13% of clean test windows and 9–13% of calibration windows were scored with 1000 continuations.

The outcome is **NOT CONFIRMED (validity)**. The validity condition fails, and it fails through the unconditional

TABLE VI

TEST DAYS: POWER AT $\alpha = 0.05$ PER SYMBOL (MEAN OVER EACH FAMILY'S STRENGTH GRID), THE POOLED DIFFERENCE WITH THE 95% INTERVAL OF THE DECISION RULE, AND THE REALISED FALSE-ALARM RATE ON CLEAN TEST WINDOWS. COND.: GENERATOR-CONDITIONAL NULL; UNCOND.: UNCONDITIONAL NULL.

	BTC	ETH	SOL	XRP	DOGE	LTC	Mean
Spoofing, cond.	0.65	0.51	0.50	0.71	0.41	0.55	0.556
uncond.	0.43	0.25	0.44	0.73	0.42	0.56	0.473
difference			95% interval 0.079 to 0.087				0.083
Quote stuffing, cond.	0.23	0.20	0.26	0.19	0.19	0.33	0.234
uncond.	0.13	0.18	0.21	0.10	0.26	0.27	0.193
difference			95% interval 0.039 to 0.045				0.042
Momentum ignition, cond.	0.17	0.10	0.06	0.63	0.21	0.44	0.270
uncond.	0.19	0.14	0.17	0.83	0.48	0.51	0.386
difference			95% interval -0.119 to -0.112				-0.116
False alarm, cond.	0.049	0.047	0.052	0.039	0.050	0.050	0.048
uncond.	0.066	0.080	0.140	0.073	0.232	0.079	0.112

TABLE VII

TEST DAYS: REALISED FALSE-ALARM RATE AT $\alpha = 0.05$ BY TEST DAY. DAY T1 IS CALIBRATED ON VALIDATION DAY 6, EACH LATER DAY ON THE DAY BEFORE.

Test day	T1	T2	T3	T4	T5	T6
Conditional						
BTC	0.057	0.056	0.038	0.049	0.033	0.061
ETH	0.032	0.042	0.056	0.057	0.043	0.051
SOL	0.056	0.050	0.037	0.045	0.080	0.043
XRP	0.040	0.034	0.031	0.024	0.060	0.043
DOGE	0.044	0.036	0.067	0.027	0.088	0.036
LTC	0.048	0.049	0.048	0.058	0.048	0.051
Unconditional						
BTC	0.034	0.106	0.112	0.088	0.029	0.025
ETH	0.076	0.252	0.004	0.068	0.033	0.047
SOL	0.047	0.079	0.024	0.292	0.394	0.006
XRP	0.047	0.034	0.157	0.095	0.068	0.036
DOGE	0.104	0.018	0.555	0.010	0.670	0.036
LTC	0.026	0.187	0.089	0.052	0.072	0.046

detector (Table VI). Its realised false-alarm rate at $\alpha = 0.05$ is 0.140 on SOL and 0.232 on DOGE, above the 0.10 limit, and it exceeds the sampling tolerance of a rate of 0.05 on all 6 symbols. The conditional detector's rate lies between 0.039 and 0.052 and is within that tolerance on every symbol. By day (Table VII), the conditional rate never exceeds 0.088, while the unconditional rate reaches 0.670 (DOGE, day T5). The power condition would have held: the pooled difference at $\alpha = 0.05$ is 0.083 for spoofing (95% interval 0.079 to 0.087) and 0.042 for quote stuffing (0.039 to 0.045), but -0.116 for momentum ignition (-0.119 to -0.112). Under the rule this does not change the outcome. The rule asks for both detectors to be valid because a difference in raw power is only a comparison of nulls when both thresholds mean the same false-alarm rate; here the unconditional detector's threshold was looser on every symbol.

The measures the rule lists as reported but not decisive qualify the power result in both directions. The unconditional detector's raw power is inflated by its loose threshold. With each detector's threshold set so that its false-alarm rate on the clean test windows is at most 0.05, the conditional detector is ahead in all three families: 0.565 against 0.346 for spoofing, 0.238 against 0.118 for stuffing and 0.276 against 0.261 for ignition. The AUC agrees for spoofing (0.884 against 0.802)

and stuffing (0.606 against 0.553) but not for ignition (0.715 against 0.730). At $\alpha = 0.01$ the conditional detector has lower power in all three families, so the far tail remains its weak point. With Benjamini–Hochberg at $q = 0.10$ on mixtures with 5% prevalence, the conditional detector makes almost no discoveries (power at most 0.007), while the unconditional detector reaches power 0.060–0.126 at a false-discovery proportion of about 0.203, twice the target.

Exploratory analyses. The rest of this subsection was not part of the rule; we added it after the decision, and none of it changes the outcome.

Ignition reversed. On the validation days under the same design the conditional detector led (0.390 against 0.235); on the test days it is behind on all six symbols. Part of that gap is the unconditional detector's looser threshold, since at matched size the conditional detector is ahead (0.276 against 0.261), but its AUC is also lower, so its ranking of ignition windows is no better. We do not know why.

Symbols as replicates. The rule's bootstrap resamples windows within each symbol and holds the symbols fixed, so it ignores variation between symbols: the conditional detector has higher raw power for spoofing on only 3 of six symbols and for stuffing on 5. Resampling symbols as well as windows gives 95% intervals for the pooled difference of -0.006 to 0.177 for spoofing, -0.004 to 0.081 for stuffing and -0.193 to -0.050 for ignition (a t interval over the six per-symbol means is wider still). Under this analysis the power condition would not have held either. At matched size the picture is firmer: t intervals over symbols for the difference in size-matched power are 0.093 to 0.346 for spoofing and 0.091 to 0.149 for stuffing, and include zero for ignition (-0.070 to 0.098). The raw-power difference is uncertain mostly because the unconditional detector's threshold varies so much from symbol to symbol.

Where the unconditional false alarms came from. The unconditional detector exceeded 0.10 on 10 of the 36 symbol-days. On every one of them the share of events that move the mid was at least 1.6 times that of the calibration day before, against a median ratio of 0.9 on the other days. Across all symbol-days, the unconditional false-alarm rate rises with this ratio (Spearman $\rho = 0.94$) and the conditional one does not ($\rho = 0.16$, $p = 0.36$; Fig. 1). On DOGE day T5, where the mid moved on 3.9 times as many events as the day before, 0.87 of clean windows had an unconditional p-value of at most 0.01 for the number of mid moves, against 0.06 under the generator. The pooled reference set reads a busier market as unusual; the generator, simulating from each window's own busy context, does not. Symbol-days are not independent, so we do not treat the correlation as a test.

Other injection seeds. Every run so far used one seed, which fixes both where the footprints land and the generator's draws. We reran the validation days under two more seeds with the same settings and scored all 3 with V3 (Table VIII; no test data were read). The largest false-alarm rate over symbols and seeds was 0.075 for the conditional detector and 0.070 for the unconditional one. Under every seed the conditional detector had the higher power in all three families, and the higher size-matched power on at least 5, 6 and 6 of six symbols for spoofing, stuffing and ignition. For spoofing its size-matched power ranged from 0.630 to 0.650 over seeds, against 0.467–0.519 for the unconditional detector, and neither detector's AUC moved by more than 0.009 in any family. The validation result does not depend on the seed; the test days were run under one seed.

TABLE VIII

VALIDATION DAYS UNDER THREE INJECTION SEEDS, DESIGN V3, ADDED AFTER THE TEST-DAY DECISION: THE LARGEST REALISED FALSE-ALARM RATE OVER THE SIX SYMBOLS AND POWER AT $\alpha = 0.05$ (MEANS OVER STRENGTHS AND SYMBOLS). BOLD: THE HIGHER POWER OF THE PAIR. SEED 0 IS THE V3 ROW OF TABLE IV.

	Max. false alarm		Spoofing		Stuffing		Ignition	
Seed	cond.	uncond.	cond.	uncond.	cond.	uncond.	cond.	uncond.
0 (follow-up)	0.075	0.062	**0.655**	0.444	**0.248**	0.173	**0.390**	0.235
1	0.069	0.070	**0.664**	0.462	**0.247**	0.166	**0.399**	0.225
2	0.065	0.060	**0.677**	0.414	**0.262**	0.170	**0.401**	0.229

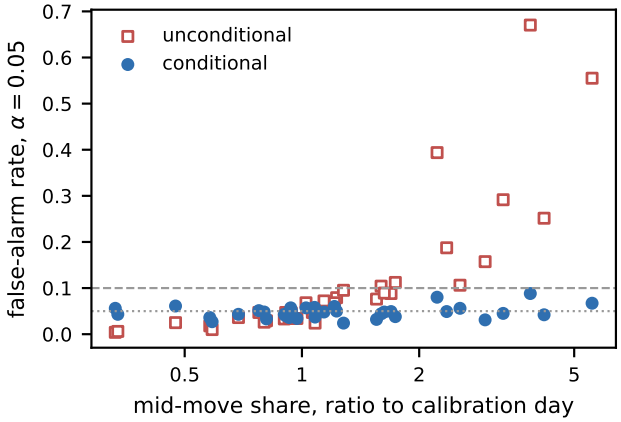


Fig. 1. Test days, exploratory: each symbol-day's realised false-alarm rate at $\alpha = 0.05$ against the ratio of its mid-move share to that of its calibration day (log scale). Dotted: nominal 0.05; dashed: the rule's limit 0.10.

VI. DISCUSSION

What the conditional null buys. Holding the statistics, the combination and the calibration fixed, replacing a pooled reference set by simulations from the window's own context raised power against all three footprints. Q1 shows that the gain is in the ranking, not in a looser threshold: at matched size and in the threshold-free AUC, the conditional detector separates injected from clean copies better. The gain is largest for spoofing and momentum ignition, whose footprints (a large same-price queue jump, fast mid moves) are large or small only relative to the current state of the book. Quote stuffing is hard for both detectors (AUC 0.622 and 0.567), which says more about the panel than about the null: one-lot flickers of a best queue barely move any statistic in Table I. The test days support this reading for spoofing and stuffing, where the conditional detector kept the higher AUC and the higher size-matched power, also with symbols treated as replicates, but not for ignition: there its AUC and raw power were lower, and its size-matched power was no different. Because the run failed on validity, the rule confirms the gain for no family.

Where it is weak. The advantage thins in the far tail. At size-matched $\alpha = 0.01$ the conditional detector falls behind for spoofing and stuffing. Its p-values cannot go below $1/(M+1)$, and only windows that reach the floor get 1000 continuations, while the unconditional detector always compares with 1500 reference windows. On the test days it was behind in all three families at $\alpha = 0.01$ and made almost no Benjamini–Hochberg discoveries. More continuations would help at linear cost: Stage 0 already took 1.6–2.3 hours per symbol on a cluster node.

Exchangeability is a property of the whole pipeline. Conformal validity needs calibration and evaluation windows to be scored by the same procedure. Our refinement rule satisfied this on paper and broke it in practice, because a clean window's resolution depended on its injected siblings. Any adaptive step in scoring (extra simulation, early stopping, cached neighbours) must depend only on the window being scored. The drift result makes a second point: a single calibration day is a fragile reference even for a model that conditions on context, because the generator's errors change from day to day. Day-before calibration is the simplest fix, and on the test days it was enough for the conditional detector but not for the unconditional one, which exceeded the limit on SOL and DOGE. The exploratory analysis of Section V-C points to why: the unconditional false alarms came on days when the market became much busier than the day before, and conditioning on the window's context absorbed that change (Fig. 1). This is one run on six symbols, and the analysis was not planned, so it is a hypothesis for the next pre-registered run rather than a finding. Weighted conformal methods [18], [19] and online adjustment of the level [20] are principled alternatives that we have not tried.

Operational cost. Rolling calibration assumes that yesterday was clean. Manipulated windows in the calibration set raise calibration scores and so make the test conservative rather than liberal. The price is lost power, not false alarms.

VII. Limitations

- **One confirmatory run.** The design was built on validation days and tested once, on six test days. That run was not a confirmation, and the reasons it gives (unconditional false alarms, the ignition reversal) rest on six symbols and six days. Its bootstrap ignores variation between symbols; with symbols resampled too, the raw power differences for spoofing and stuffing are not distinguishable from zero. The analyses that explain the outcome were added after it.
- **Synthetic labels.** Injected footprints are a stand-in for manipulation. Power against real manipulation is unknown, and the study detects no real case.
- **Passive injection.** The market does not react to the injected orders. A real spoofer's effect on other traders is not modelled.
- **Level 1 only.** Layering at depth and schemes that span venues are invisible here.
- **Repair chosen on the same data.** The follow-up rule was written after the Stage-0 result was known, and its repaired design was selected on the validation days it is evaluated on. On the test days the conditional detector's false-alarm control held, but the comparison as a whole was not confirmed.
- **One month, six symbols, one test-day seed.** The comparison rests on February 2024 crypto futures. Two further injection seeds left the validation result unchanged (Table VIII), but the test days were run under one seed.

VIII. Conclusion

A fitted order book generator can serve as the null of a surveillance test. On validation days of six crypto futures contracts, conditioning the null on each window's context raised power against three injected footprints under the same conformal calibration, and the gain survived matching the false-alarm rate at $\alpha = 0.05$. The pre-registered comparison still failed, on validity: a refinement rule that looked at injected copies and a single calibration day made the conditional detector flag clean windows too often. Matching the refinement and calibrating on the day before keeps every symbol within the limit and keeps the power advantage, under each of three injection seeds. That repair was found on the same validation days on which it is judged, so we ran it once on six unread test days under a rule committed beforehand. The result was NOT CONFIRMED. The conditional detector stayed within the false-alarm limit on every symbol and day; the unconditional one did not, and the rule required both. Power was higher for spoofing and stuffing and lower for ignition, and the conditional detector stays weak at small α. On these test days a generator-conditional null kept its false-alarm rate within the limit under day-to-day drift where a pooled null did not; that it is also more powerful holds at matched false-alarm rates for two of three footprints across symbols, in an analysis added after the run, and is not confirmed.

Acknowledgment

The author thanks Sardar Vallabhbhai National Institute of Technology (SVNIT), Surat, for access to its high-performance computing cluster.

Declaration of AI use

The author used Claude (Anthropic), an AI assistant, to help write and test the analysis code, to check the references and to help draft and edit the text. The author reviewed and edited the content and takes full responsibility for it.

References

[1] V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*. New York, NY, USA: Springer, 2005.

[2] S. Bates, E. Candès, L. Lei, Y. Romano, and M. Sesia, "Testing for outliers with conformal p-values," *Ann. Statist.*, vol. 51, no. 1, pp. 149–178, 2023.

[3] A. Khatib, "Noise models matter more than dynamics in synthetic level-1 order flow," preprint, 2026. [Online]. Available: https://github.com/Arshad-13/orderflow-noise-models

[4] T. J. Putniņš, "Market manipulation: A survey," *J. Econ. Surveys*, vol. 26, no. 5, pp. 952–967, 2012.

[5] Á. Cartea, S. Jaimungal, and Y. Wang, "Spoofing and price manipulation in order-driven markets," *Appl. Math. Finance*, vol. 27, no. 1–2, pp. 67–98, 2020.

[6] E. J. Lee, K. S. Eom, and K. S. Park, "Microstructure-based manipulation: Strategic behavior and performance of spoofing traders," *J. Financial Markets*, vol. 16, no. 2, pp. 227–252, 2013.
[7] X. Wang and M. P. Wellman, "Spoofing the limit order book: An agent-based model," in *Proc. 16th Int. Conf. Autonomous Agents and Multiagent Systems (AAMAS)*, 2017.
[8] J. F. Egginton, B. F. Van Ness, and R. A. Van Ness, "Quote stuffing," *Financial Management*, vol. 45, no. 3, pp. 583–608, 2016.
[9] Y. Cao, Y. Li, S. Coleman, A. Belatreche, and T. M. McGinnity, "Adaptive hidden Markov model with anomaly states for price manipulation detection," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 26, no. 2, pp. 318–330, 2015.
[10] X. Tao, A. Day, L. Ling, and S. Drapeau, "On detecting spoofing strategies in high-frequency trading," *Quant. Finance*, vol. 22, no. 8, pp. 1405–1425, 2022.
[11] K. Kularatnam and T. Stathaki, "Detecting and triaging spoofing using temporal convolutional networks," arXiv:2403.13429, 2024.
[12] Y. Lin and P. Yang, "Detecting multilevel manipulation from limit order book via cascaded contrastive representation learning," arXiv:2508.17086, 2025.
[13] T. Fabre and D. Challet, "Learning the spoofability of limit order books with interpretable probabilistic neural networks," arXiv:2504.15908, 2025.
[14] S. Tiwari, H. Ramampiaro, and H. Langseth, "Machine learning in financial market surveillance: A survey," *IEEE Access*, vol. 9, pp. 159734–159754, 2021.
[15] A. Coletta, M. Prata, M. Conti, E. Mercanti, N. Bartolini, A. Moulin, S. Vyetrenko, and T. Balch, "Towards realistic market simulations: A generative adversarial networks approach," in *Proc. 2nd ACM Int. Conf. AI in Finance (ICAIF)*, 2021, pp. 1–9.
[16] A. Coletta, J. Jerome, R. Savani, and S. Vyetrenko, "Conditional generators for limit order book environments: Explainability, challenges, and robustness," in *Proc. 4th ACM Int. Conf. AI in Finance (ICAIF)*, 2023, pp. 27–35.
[17] Y. Benjamini and Y. Hochberg, "Controlling the false discovery rate: A practical and powerful approach to multiple testing," *J. R. Stat. Soc. B*, vol. 57, no. 1, pp. 289–300, 1995.
[18] R. J. Tibshirani, R. Foygel Barber, E. Candès, and A. Ramdas, "Conformal prediction under covariate shift," in *Advances in Neural Information Processing Systems 32*, 2019.
[19] R. Foygel Barber, E. J. Candès, A. Ramdas, and R. J. Tibshirani, "Conformal prediction beyond exchangeability," *Ann. Statist.*, vol. 51, no. 2, pp. 816–845, 2023.
[20] I. Gibbs and E. Candès, "Adaptive conformal inference under distribution shift," in *Advances in Neural Information Processing Systems 34*, 2021.
[21] Y. Liu and J. Xie, "Cauchy combination test: A powerful test with analytic p-value calculation under arbitrary dependency structures," *J. Amer. Statist. Assoc.*, vol. 115, no. 529, pp. 393–402, 2020.
[22] B. Phipson and G. K. Smyth, "Permutation p-values should never be zero: Calculating exact p-values when permutations are randomly drawn," *Stat. Appl. Genet. Mol. Biol.*, vol. 9, no. 1, Art. 39, 2010.